

Problem S1. Good Fours and Good Fives

We need to count the nonnegative integer solutions to

$$4a + 5b = N.$$

Loop over all possible values of a , from 0 to $\lfloor N/4 \rfloor$. For each a , the remaining value is $N - 4a$; it contributes one solution exactly when it is divisible by 5.

The same idea works by looping over b instead. A more arithmetic solution can exploit the period 20, but the one-loop solution is already straightforward.

Complexity. The simple solution runs in $O(N)$ time and uses $O(1)$ memory.

Problem S2. Good Groups

This is the senior version of the group-constraint task. Store the must-be-together and must-not-be-together constraints, then read the final groups and build a dictionary from student name to group number.

A must-be-together pair is violated if the two group numbers differ. A must-not-be-together pair is violated if the two group numbers are equal. The answer is the total number of violations.

Using a dictionary is important for the strongest constraints: checking membership by scanning all groups for every query is unnecessarily slow.

Complexity. The running time is linear in the input size, assuming expected $O(1)$ dictionary operations.

Problem S3. Good Samples

The construction is based on controlling how many new good samples are created when one more note is appended. All length-1 samples are always good, so initially there are N unavoidable good samples. We must create exactly $K - N$ additional good samples.

For simpler subtasks with only two pitches, no good sample can have length 3, so it is enough to decide whether each adjacent pair should be good. For the full version, keep enough information about the current suffix to know how many good samples would end at the next position for each candidate note. Greedily choose a note that creates as many new good samples as possible without exceeding the remaining target.

Equivalently, maintain the longest suffix that is currently good. When appending a value, the number of new good samples is the length of the longest good suffix ending at the new position. Choose the next value so this contribution matches the remaining count as closely as possible.

Complexity. With careful suffix maintenance, the construction can be implemented in $O(N)$ time.

Problem S4. Good Triplets

For three points on a circle, the origin lies strictly inside their triangle exactly when the three cyclic gaps between consecutive chosen points are all strictly less than half the circumference. After sorting the positions, this condition can be checked by comparing arc lengths.

A direct $O(N^3)$ triple loop is enough only for the first subtask. For a faster solution, sort the points around the circle and duplicate the array with each position plus C . For each starting point, use a two-pointer window to count how many following points lie within a semicircle. Triples that fit inside some semicircle are bad; subtract the number of bad triples from the total number of triples.

When several points share a position, store multiplicities and use prefix sums or combinatorial counts so that identical positions are handled correctly.

Complexity. Sorting gives $O(N \log N)$ time; the two-pointer scan is linear after sorting.

Problem S5. Good Influencers

Root the friendship tree at an arbitrary student. The goal is to make every student intend to write the CCC while minimizing cost, and the interaction between a vertex and its parent is the only information that must cross a subtree boundary.

Use a tree DP with three states for each vertex v :

- $dp[v][0]$: v will be influenced by its parent;
- $dp[v][1]$: v has no special interaction with its parent;
- $dp[v][2]$: v will influence its parent.

For each vertex, combine the DP values of its children and account for whether v initially intends to write the CCC. The answer is $dp[root][1]$, because the root has no parent.

The naive combination can be quadratic if every transition scans many children repeatedly. By deriving the minimum child contributions for each state and merging children once, the DP can be optimized to linear time.

Complexity. The optimized tree DP runs in $O(N)$ time and uses $O(N)$ memory.